

Session 8 Speaker Script — COMPLETE WITH FULL TALKING POINTS

Scaling & Production Deployment

Slides 8-14 (Continuation)

08. HANDLING FAILURES

~8 minutes

WHAT'S ON SCREEN

Failure scenarios • Circuit breakers • Graceful degradation • Bulkheads • Exponential backoff • Real examples

SPEAKING SCRIPT

At scale, failures aren't if. They're when. How do you handle them?

FAILURE TYPES:

- 1) SERVICE CRASH: AI model process crashes. That server stops handling requests.
- 2) DATABASE DOWN: Database becomes unreachable. All queries fail.
- 3) EXTERNAL API TIMEOUT: You call external service (e.g., payment processor). It's slow or down.
- 4) RESOURCE EXHAUSTION: Server runs out of memory. Starts dropping requests or crashing.
- 5) CASCADING: One service fails. Causes downstream services to fail. Soon everything is down.
- 6) SLOW DEGRADATION: System doesn't crash but gets slower and slower. Eventually unusable.

STRATEGY 1: CIRCUIT BREAKER

Pattern: When calling another service (database, external API), monitor for failures. If failure rate gets high, STOP sending requests temporarily. Let the service recover. Then try again.

States:

- CLOSED (normal): Requests flowing, failures rare
- OPEN (broken): Too many failures. Stop sending requests immediately. Fail fast.
- HALF-OPEN (recovering): Service had problems. Try sending requests again. If work, go CLOSED. If still failing, go OPEN again.

Example: Calling payment processor API

- Requests working fine (CLOSED state)
- Payment processor gets slow. 30% of requests timeout.
- Circuit breaker detects failure rate > 20%, opens circuit
- Now, don't even try calling payment processor. Fail immediately, tell user "payment unavailable temporarily"
- After 30 seconds, try one request (HALF-OPEN). Works? Good, close circuit. Doesn't work? Stay open.

Benefit: Prevents cascading failures. Stops wasting time on failed requests. Gives failing service time to recover.

STRATEGY 2: GRACEFUL DEGRADATION

System not working perfectly? Give users something rather than nothing.

Example 1 (Hiring AI):

- Ideal: Use full AI system (fine-tuned model + RAG + everything)
- If AI model is slow: Skip RAG, just use fine-tuned model (faster)
- If fine-tuned model is down: Use simple rule-based system (good candidates = 5+ years experience in domain)
- If everything fails: Show generic "we're experiencing issues, please try later"

Example 2 (Netflix):

- Ideal: Personalized recommendations using complex ML model
- If recommendation system slow: Show popular content (pre-computed, fast)■r/>- If everything fails: Show trending now (static, cached)

Key: User gets SOMETHING. System still works, just reduced functionality.■r/>
STRATEGY 3: BULKHEADS

Isolate failures. Don't let one failure take down entire system.

Example: Your system handles hiring recommendations AND resume screening (two different features).

- Hiring recommendation service: 50 servers■r/>- Resume screening service: 50 servers
- Database connection pool: Separate pools for each service (not shared)
- Message queue: Separate queues for each service

If resume screening crashes: Hiring recommendations keep working. Services are isolated (bulkheads).

Without bulkheads: If resume screening uses all database connections, hiring recommendations have none. Both services fail.

STRATEGY 4: EXPONENTIAL BACKOFF

When retrying failed requests, don't retry immediately. Wait longer each time.

Pattern:

- First retry: Wait 1 second, try again
- Second retry: Wait 2 seconds, try again
- Third retry: Wait 4 seconds, try again
- Fourth retry: Wait 8 seconds, try again
- Stop after N retries

Add randomness (jitter): Instead of exactly 1 second, wait 0.5-1.5 seconds. This prevents thundering herd (all clients retrying simultaneously).

Why: If service is overloaded, hammering it with retries makes it worse. Backing off gives it time to recover.

STRATEGY 5: TIMEOUTS

Don't wait forever for response. Set timeout. If no response by timeout, fail and move on.

Example: Calling database. Set timeout to 5 seconds. If query takes > 5 seconds, return error instead of hanging.

Why: Without timeout, request can hang forever. Server resources get exhausted (connections pile up).

STRATEGY 6: BULKS AND RATE LIMITING

Protect from being overwhelmed.

Rate limiting: Limit requests from single user/IP. E.g., max 100 requests per minute. After that, reject requests.

Why: Prevents single user (or attacker) from overwhelming system.

Bulk splitting: If many requests come in at once, process them in batches. Don't try to do all at once (would exhaust resources). Process N at a time.

REAL EXAMPLE: UBER DURING SURGE PRICING

Scenario: Major event ends. 100K people simultaneously request rides.

System design:

1. Request rate limiter: Accept 10K requests/second. Queue rest.
2. Matching service: Process queued requests, match drivers to riders.■r/>3. If matching service gets slow: Bulkhead

protection means recommendation system (showing suggested prices) keeps working.■r/>4. If payment processor times out: Still show ride offer to user, process payment asynchronously later.■r/>5. If Redis cache (caching driver locations) goes down: Fall back to database (slower but still works).■r/>6. Every 30 seconds, system checks: Are critical services healthy? If not, automatically degrade functionality.■r/>
Result: System handles surge without crashing. Users might see slightly degraded experience but service is available.

FAILURE RECOVERY PROCEDURE:

When failure detected:

1. Alert on-call team
2. Attempt automated recovery (restart service, failover to replica)
3. If manual intervention needed: On-call person takes action
4. Implement fix or workaround
5. Monitor that system is recovering
6. Post-incident review: How do we prevent this next time?

CHAOS ENGINEERING:

Proactively test failure scenarios. Netflix practices this at scale. They have system called Chaos Monkey that randomly kills servers in production to test system resilience.

Idea: If system can survive random server deaths, it's probably robust.

Others: Chaos Kong (entire data center failure), Chaos Gorilla (AZ failure).

While extreme, principle is good: Test failures before they happen in uncontrolled ways.

DELIVERY TIPS

These are real techniques used in production systems. Make them concrete. • Circuit breaker is powerful pattern—show how it prevents cascading. • Graceful degradation is user-focused—emphasize: something is better than nothing. • Bulkheads prevent one failure from becoming total system failure. • Uber example shows how multiple strategies work together. • Chaos engineering shows maturity: proactively testing for failures.

09. REAL-WORLD SCALING EXAMPLES

~8 minutes

WHAT'S ON SCREEN

5 Companies at scale • Google Search, Netflix, Uber, Meta, Tesla • Architecture, challenges, solutions • Numbers: queries, users, predictions

SPEAKING SCRIPT

Let me show you how real companies scaled their systems to handle impossible-seeming loads.

EXAMPLE 1: GOOGLE SEARCH

Scale: 8.5 billion searches per day = 100K searches per second

Architecture:

- Front-end: Thousands of datacenters worldwide
- Each datacenter: Thousands of machines
- Index (inverted mapping of words to documents): Stored across multiple datacenters, sharded
- Ranking model: Hundreds of signals fed to ML model to rank results
- Cache layer: Previous popular queries cached

Challenges:

- Latency: Users expect results in < 100ms. Need to query millions of documents, rank them, format results ALL in < 100ms.
- Consistency: New pages added to web constantly. Search index must be updated. Need freshness vs consistency tradeoff.
- Sharding: Index is too large for single machine. Split across thousands. Coordinating all these shards is complex.

Solutions:

- Pre-computation: Popular queries pre-ranked and cached
- Approximation: Don't search entire index. Use bloom filters and other techniques to prune search space
- Distributed computation: Parallel processing across thousands of machines
- Redundancy: Every piece of data replicated 3+ times in case of failure
- Monitoring: Thousands of metrics, automated alerts for any anomalies

Result: Handles 100K queries/second reliably. If one datacenter fails, others take over. System is resilient and fast.

EXAMPLE 2: NETFLIX

Scale: 250 million users, 200 million hours watched per day

Architecture:

- Recommendation engine: ML model predicts what user wants to watch next. Most critical system.
- Personalization: Different UI for different users (based on their preferences)
- Video delivery: Stream video to users globally (CDN—content delivery network)
- A/B testing framework: Test new algorithms before shipping

Challenges:

- Personalization at scale: Computing 250M different personalized UIs is hard
- Recommendation accuracy: Even 1% improvement in recommendations = millions more hours watched = millions more revenue
- Latency: Recommendation must come back in < 100ms for good user experience
- Freshness: New movies added daily. Recommendations must reflect new content

Solutions:

- Pre-computing: Pre-compute recommendations offline for most users. Serve cached results online.
- Multi-stage ranking: First stage (fast, approximate) narrows from millions of titles to thousands. Second stage (slower, accurate) ranks those thousands.
- Real-time personalization: For some users, compute recommendations in real-time (Tier 1 users get better experience)
- Continuous learning: Feed user behavior back into model. Recommendations improve over time.

- Microservices: Break system into independent services. Recommendation service, personalization service, video delivery service, etc. Each scales independently.

Result: 250M users get personalized experience. Recommendations are good (users watch what Netflix recommends). System handles spikes (new season release = 10M simultaneous watchers).

EXAMPLE 3: UBER

Scale: 100 million active users. Millions of drivers. Millions of simultaneous ride requests at peak hours.

Key challenges:

- Real-time matching: When user requests ride, match to nearest available driver. Must happen in < 10 seconds.
- Geospatial: Need fast queries like "find all drivers within 2 km of user". Naive approach is too slow.■r/>- Surge pricing: Dynamically adjust prices based on demand. More requests = higher price. Less requests = lower price.
- Consistency: All riders and drivers globally see consistent prices/availability (with some regions seeing different prices due to local demand).

Architecture:

- Geospatial indexing: Use specialized spatial indexing (quadtrees, H3 hexagons) to quickly find nearby drivers
- Event streaming: Use Kafka to stream driver location updates. When driver moves, update their location.
- Real-time pricing: ML model predicts demand, adjusts surge pricing dynamically
- Prediction service: Predict where demand will be in next 30 minutes. Suggest to drivers where to go.■r/>- Dispatch service: Match riders to drivers using algorithms (optimize for speed, fairness, distance)■r/>

Challenges at scale:

- During major event (stadium lets out): 100K people simultaneously request rides. System must handle without crashing.■r/>- Geographic regions: US, India, China all using Uber simultaneously with local regulations, currencies, languages.■r/>- Peak hours: Morning rush 7-9am, evening rush 5-7pm, nightlife weekend. Traffic in system varies 10x between peak and off-peak.

Solutions:

- Auto-scaling: Automatically add servers during peak hours
- Geospatial sharding: Split drivers by region. US service handles US drivers, India service handles India drivers, etc.■r/>- Feature flags: Can disable features if necessary (e.g., if matching service overloaded, disable surge pricing, just charge flat rate)
- Rate limiting: Accept only 10K rides/second. Queue rest. Process from queue as capacity available.■r/>

Result: Service is available 99.99% of the time. Users can request rides anytime. Drivers are efficiently matched. System handles peaks without crashing.

EXAMPLE 4: META (FACEBOOK)

Scale: 3 billion users. 1 trillion objects in database. Trillions of relationships (friendships).

Challenge: Feed ranking. What posts should show in your feed?

Solution: Complex ranking model (thousands of signals fed to neural network):

- Does this post come from friend? (strong signal)
- What topic? (shows interest in topic?)
- Recency (newer is better)
- Engagement (do my friends like this?)
- Diversity (don't show 10 similar posts)

ML model ingests these signals, outputs: should this post be ranked high in this user's feed?

Scale: For 1 billion active users daily, 300M posts created daily, you need to rank: 300M posts for 1B users = 300 trillion ranking decisions per day.

Can't do this online (too slow). So:

- Offline: Batch process. Compute scores for all posts and all users overnight.■r/>- Online: Serve pre-computed scores, with some real-time adjustment
- Users: Get feed of top-ranked posts within 100ms

Result: Feed shows relevant content. Users stay engaged. System handles massive scale.

EXAMPLE 5: TESLA AUTONOMOUS DRIVING

Scale: 4 million cars collecting terabytes of video per day. Each car's on-device AI makes thousands of predictions per second (object detection, lane detection, etc).

Challenge: Real-time inference at edge (on the car, not in datacenter). Can't have 500ms latency (car is moving at 100km/h).

Architecture:

- Models are small enough to run on car's GPU (optimized inference)
- Predictions happen every 50ms (20 times per second)
- Results sent to datacenter for logging and improvement
- Model updates happen overnight (car downloads new model when parked)

Scale challenges:

- Millions of cars, each producing unique scenarios. Training data is constantly growing.
- Safety critical (mistakes can cause accidents). Need very high accuracy.■r/>- Hardware varies: Older cars have less powerful GPUs. New cars have more powerful GPUs. Models must work on all.

Solutions:

- Telemetry: Every edge case is logged. Used to improve future models.■r/>- Model compression: Keep models small enough to fit on car hardware■r/>- Continuous improvement: Every week, new model trained, tested thoroughly, deployed to cars■r/>- Fallback: If confidence < threshold, don't make autonomous decision. Alert driver.■r/> Result: Cars drive themselves safely most of the time. Edge cases trigger driver alerts. System improves weekly based on real-world data.

PATTERNS ACROSS ALL 5:

1. Pre-computation: Expensive computations done offline, results served online
2. Approximation: Don't need perfect answers, close enough answers are faster
3. Caching: Popular queries cached, served instantly
4. Distributed: Work split across thousands of machines
5. Monitoring: Everything is monitored, anomalies trigger alerts
6. Graceful degradation: If something fails, system still works (reduced functionality)
7. Continuous improvement: Systems learn from real-world usage

DELIVERY TIPS

Real companies at real scale. These are not hypothetical. • Numbers are important: 8.5B searches, 250M users, 100K rides/sec. • Each example has specific challenges and solutions. Show how principles from session 8 apply. • Tesla example shows edge computing (not just cloud)—expands thinking. • Patterns section ties all 5 together—students see commonalities.

10. COST & OPTIMIZATION

~5 minutes

WHAT'S ON SCREEN

Cost breakdown • Infrastructure, storage, bandwidth, people • Optimization strategies • Cost-benefit analysis • Trillion-dollar question: Scale vs Spend

SPEAKING SCRIPT

At scale, cost becomes a real constraint. You can't just throw infinite compute at problem.

COST BREAKDOWN:

Assume hiring AI system at scale: 1 million requests per day

Infrastructure: \$50K/month

- 100 servers × \$500/month: \$50K
- Database: \$10K
- Load balancers: \$2K
- Storage: \$5K
- Total: \$67K

Data & Storage: \$20K/month

- Logs storage (terabytes per month): \$10K
- Database backups: \$5K
- Model storage and versioning: \$5K

Bandwidth: \$15K/month

- Data transfer out (1M requests × 1KB response × \$0.15/GB): \$150/month (small) but if you include training data transfers, backups, etc: \$15K

Observability: \$10K/month

- Monitoring tools (Datadog, etc): \$5K
- Logging storage: \$5K

People: \$200K/month

- 2 ML engineers @ \$5K/month each: \$10K
- 1 infrastructure engineer: \$5K
- 1 data engineer: \$5K
- 1 on-call engineer (rotation): \$5K
- 1 manager: \$5K
- Fringe benefits (insurance, taxes, office): 1.5x salary multiplier: \$35K
- Total: \$65K
- But this is small team. For 1M requests/day at a bigger company: multiply by 10.

Total monthly cost: ~\$300K

Annual: ~\$3.6M

OPTIMIZATION STRATEGIES:

1) CACHING

Problem: Database queries are expensive (CPU, I/O).

Solution: Cache results. Hiring recommendation for candidate X cached for 1 hour. Second request for same candidate uses cache (instant, no database query).

Savings: 80% of requests hit cache. Reduces database load 5x. Can reduce database servers by 50%.

Cost savings: \$5K/month

2) BATCHING

Problem: Individual requests have overhead.

Solution: Batch 100 requests together, process as one.

Benefit: Overhead amortized across 100 requests.

Savings: \$3K/month (fewer servers needed)

3) MODEL OPTIMIZATION

Problem: AI model inference takes 2 seconds per request, costs a lot of compute.

Solution: Distill model (create smaller model that's almost as accurate, runs faster). Or use approximations.

Benefit: 10x faster inference. 10x fewer servers needed.

Cost savings: \$50K/month

4) REGIONAL OPTIMIZATION

Problem: Serving globally is expensive. Cloud costs vary by region. US East Coast expensive. Asia cheap. Solution: Route requests to cheapest region that meets latency requirements. Bigger computation in cheap regions, results cached closer to users.

Cost savings: \$20K/month

5) RESOURCE MONITORING

Problem: Servers are idle during off-peak hours but still cost money.

Solution: Use auto-scaling. During peak hours (7-9am, 5-7pm), run 100 servers. Off-peak: run 20 servers.

Cost savings: 50% reduction during off-peak hours = \$25K/month

TOTAL OPTIMIZATION: \$103K/month = \$1.2M/year

This is why companies hire optimization engineers. Every 1% efficiency = \$30K/month. At scale, optimization pays for itself many times over.

TRILLION DOLLAR QUESTION:

You can scale system 10 different ways. Each has different cost-performance tradeoff.

- Option A: \$1M/month, handles 1M requests/sec, 50ms latency
- Option B: \$2M/month, handles 1M requests/sec, 10ms latency
- Option C: \$0.5M/month, handles 100K requests/sec, 100ms latency

Which to choose?

Depends on business model:

- If revenue per user is high and users are price-sensitive to latency (trading systems, ride-sharing): Choose B. Faster = more users.
- If revenue per user is low and users are latency-tolerant: Choose A or C. Cost matters more than speed.
- If you're just starting and uncertain about traffic: Choose C. Add capacity as traffic grows.

THE COST CURVE:

For hiring AI system:

- Cost increases non-linearly with scale
- 1M requests/day: \$300K/month
- 10M requests/day: \$2M/month (not 10x, probably 6-7x due to economies of scale)
- 100M requests/day: \$15M/month
- 1B requests/day: \$100M/month

At some point, cost per request plateaus as you've optimized everything possible.

MONETIZATION:

If hiring AI charges customers: B2B (selling to companies): Might charge \$10-100 per recommendation

- \$0.5 revenue per request x 1M requests = \$500K/month (exceeds cost of \$300K)
- Profitable at scale

If free service (supported by ads/data):

- Need 50M+ requests/month to make economics work (advertising revenue covers cost)
- Or need to reduce cost through optimization

DELIVERY TIPS

Cost is often overlooked by students. Make it real. • Detailed breakdown of \$300K/month shows all cost categories. • Optimization strategies are concrete: show \$1-50K savings per approach. • Trillion-dollar question is actually trillion-dollar question at Meta, Google scale. • Economics matter: system must be profitable or it shuts down. • Resource monitoring and auto-scaling save significant money.

11. TEAM & PROCESSES AT SCALE

~5 minutes

WHAT'S ON SCREEN

Team structure • Roles needed at scale • Communication and processes • Incident management • On-call culture

SPEAKING SCRIPT

You can't scale a system without scaling the organization that builds it.

SMALL TEAM (Startup)

3 people: 1 backend engineer, 1 frontend engineer, 1 ML engineer

- Everyone knows entire system
- Everyone can deploy
- Everyone on-call
- Decision-making: Quick, informal
- Meetings: Rare

Works for: < 1M requests/day

MEDIUM TEAM (Scale-up)

15 people: 4 ML engineers, 4 backend engineers, 3 frontend engineers, 2 data engineers, 1 product manager, 1 ops engineer

- Split by component: Recommendation team, infrastructure team, frontend team
- Each team owns their component
- Deployment: ML team deploys models, backend team deploys services
- On-call: Engineers rotate on-call for their component
- Decision-making: Team leads meet to make decisions
- Meetings: Weekly syncs per team, monthly all-hands

Works for: 10M-100M requests/day

LARGE ORGANIZATION (Scale)

100+ people: Multiple teams per component, dedicated ops, incident command

- Model team: Experiment, train, evaluate models
- Infrastructure team: Manages servers, databases, caches
- Reliability team (SRE): Focuses on reliability, monitoring, incident response
- Data team: Data pipelines, data warehouse
- Product: Decides features
- Each team has multiple sub-teams
- Leadership layer: Team leads, managers, director
- On-call: Dedicated on-call rotation per team. On-call engineer handles issues in their domain.
- Processes: Formal change management, incident command structure, post-mortems
- Meetings: Daily standups per team, weekly syncs across teams, monthly all-hands

Works for: 1B+ requests/day

CRITICAL ROLES AT SCALE:

ON-CALL ENGINEER

Responsible for responding to alerts and incidents during their shift (typically 1 week per month).

- Must respond within 5-10 minutes
- Must have context to debug and make decisions
- Must be able to escalate to more senior people if needed
- Job is NOT to fix everything; job is to stabilize system, start investigation

SRE (SITE RELIABILITY ENGINEER)

Focuses on: reliability, monitoring, incident response, on-call rotations, post-mortems. Differs from software engineers who focus on building features. SRE focused on: making sure system doesn't break.

INCIDENT COMMANDER

During major incident, someone takes command. Not the most senior person, but person who's trained in incident command. Job: Coordinate people, decide what to try, make decisions, communicate updates.

PROCESSES:

CHANGE MANAGEMENT

Before deploying anything: 1. Code review: Another engineer reviews code 2. Automated tests: Must pass 3. Staging test: Deploy to staging environment (copy of production, not for users), verify it works 4. Deployment approval: Need approval from team lead or ops 5. Canary deployment: Deploy to 1%, 5%, 50%, 100% with monitoring 6. Post-deployment: Monitor for 24 hours

Takes time, but prevents catastrophic failures.

INCIDENT MANAGEMENT

When alert triggers:

1. Page on-call engineer 2. On-call responds, assesses severity 3. If P1 (critical): Page incident commander, page more engineers as needed 4. Incident commander: Declare incident, coordinate response 5. Investigation: On-call and engineers investigate cause 6. Mitigation: Stop the bleeding (temporary fix, stabilize system) 7. Resolution: Fix root cause 8. Post-mortem: Within 24 hours, team writes report on what happened and how to prevent

POST-MORTEM CULTURE

After incident, team meets. Discussed what happened, why, how to prevent next time.

KEY: Blameless. Goal is not to blame person who made mistake. Goal is to learn from system failures.

Good post-mortem: "Alert rule was tuned wrong, so incident wasn't detected for 30 minutes. Solution: Improve alert rules." Bad post-mortem: "Engineer didn't notice the alert. Solution: Fire engineer."

ON-CALL CULTURE

On-call is valuable experience (engineers learn resilience, troubleshooting). But also stressful (can get paged at 3am).

Good companies: Pay on-call engineers extra (\$1-2K per week on-call). Limit pages per week per person. Compensate time spent on incidents.

Bad companies: No extra pay, constant pages, no compensation for incident response. Bad on-call culture leads to: Burnout, engineers leaving, low morale.

COMMUNICATION

At scale, must be intentional about communication:

- Slack channels: #incidents, #deployments, #outages (everyone sees what's happening)
- Status page: Public-facing page showing system health
- Emails: Major updates emailed to stakeholders
- Meetings: Incident updates as things develop

DELIVERY TIPS

Organization structure reflects system architecture. As system grows, team grows. • On-call engineer role is critical—they're on the front lines. • SRE role is often misunderstood—explain it's about reliability, not support. • Process discussion shows maturity: good companies have formal processes. • Post-mortem blamelessness is important—sets culture of learning. • On-call culture affects retention—expensive engineers leave if burned out.

12. LAB EXERCISE OVERVIEW

~3 minutes

WHAT'S ON SCREEN

Lab title: 'Design Deployment Strategy' • Steps: Architecture design, Deployment choice, Monitoring plan, Failure scenarios, Cost estimate

SPEAKING SCRIPT

Alright, lab time. You're going to design a production deployment strategy for your hiring AI system.

YOUR TASK: Design how you would deploy your system for 1 million users, 100K requests/day. Make real decisions: infrastructure, deployment strategy, monitoring, cost.

STEP 1: ARCHITECTURE DESIGN (2 min)

Design infrastructure: Load balancer, web servers, database, caching, message queues. How many of each? Why?

STEP 2: DEPLOYMENT STRATEGY (1 min)

Choose: Canary, blue-green, A/B test, rolling, or combination? Why?

STEP 3: MONITORING PLAN (1 min)

What metrics track? Alert thresholds? Dashboard design? On-call process?

STEP 4: FAILURE SCENARIOS (2 min)

Describe 3 failure modes. How does system handle? Graceful degradation? Circuit breakers? Rollback?

STEP 5: COST ESTIMATE (1 min)

How much per month? Infrastructure, storage, people, etc?

STEP 6: OPTIMIZATION (2 min)

If costs too high, what optimizations? Caching strategy? Model optimization? Regional routing?

STEP 7: TEAM STRUCTURE (1 min)

Who do you need? Roles? On-call rotation?

STEP 8: LAUNCH PLAN (1 min)

How do you go from small (thousands of users) to large (millions)? Gradual or rapid?

DELIVERABLES:

1. Architecture diagram (boxes and arrows)■r/>2. Deployment strategy document (which approach, why, timeline)■r/>3. Monitoring metrics list■r/>4. Failure scenarios and responses■r/>5. Cost breakdown■r/>6. Optimization proposals■r/>7. Team structure and roles■r/>8. Launch timeline from small to large scale

DELIVERY TIPS

This lab brings together everything from Session 8. • Real system design decisions. • Lab makes them think about practical tradeoffs. • Architecture diagram is good tangible output.

13. KEY TAKEAWAYS FROM SESSION 8

~5 minutes

WHAT'S ON SCREEN

6 Core Principles • Lab vs Production, Architecture, Deployment, Monitoring, Failure handling, Team & Processes

SPEAKING SCRIPT

Takeaway One: Lab and Production Are Fundamentally Different

Lab: One machine, one user, failures are annoying but not catastrophic.

Production: Thousands of machines, millions of users, failures are business-critical.

Takeaway Two: Architecture Must Handle Scale

Cannot just scale up code. Need to rethink architecture. Caching, databases, load balancing, microservices.

Takeaway Three: Deployment Risk Decreases With Caution

All-at-once: Fast, risky. Canary: Slow, safe. Choose based on risk tolerance and blast radius.

Takeaway Four: Monitoring Is Continuous Vigilance

Golden signals, automated alerts, on-call rotation, dashboards. System is being watched 24/7.

Takeaway Five: Failures Are Inevitable; Handling Them Is Design

Circuit breakers, graceful degradation, bulkheads, timeouts. System must fail gracefully.

Takeaway Six: Scale Requires Organizational Scale

Not just infrastructure. Need teams, processes, incident management, on-call culture. Organization grows with system.

DELIVERY TIPS

These 6 principles connect Sessions 1-8 to production reality. • Emphasis on inevitability of failures (not if, when). • Organizational scaling is often overlooked—important to mention. • These aren't optional; they're requirements for production systems.

14. COMING UP NEXT

~2 minutes

WHAT'S ON SCREEN

Dark blue background • Title: 'Session 9: Advanced Techniques & Optimization' • Preview bullets • Closing remarks

SPEAKING SCRIPT

Session 9 is where we dive into advanced techniques. You've built systems, evaluated them, scaled them. Now: how do you optimize them further? How do you squeeze more performance from less compute?

You'll learn: Multi-armed bandits, online learning, federated learning, differential privacy, model distillation, and more.

These are techniques real companies use to optimize systems at scale. Netflix uses multi-armed bandits for A/B testing. Google uses federated learning. Companies use differential privacy to train on sensitive data safely.

Thank you all for today. You now understand how to scale production AI systems. You understand infrastructure, deployment, monitoring, failure handling, team structure. This is professional AI engineering. See you in Session 9.

DELIVERY TIPS

Session 9 natural progression: Optimize what you've built. • Advanced techniques are practical (not academic). • End on note of mastery: students have built and scaled a system.